

The Job Sequencing and Tool Switching Problem

Part X (continued): PCF' and Branch-and-Price

A symmetry-reduced position formulation, and column-generation algorithms for PCF and PTF

Scope and status. This note continues *Part X: Position-Indexed Formulations* (10_position_formulations.tex); all notation, the free-initial-load convention, the value Z_{SSP}^* , the trajectory normal form, and the formulations PCF and PTF are taken from there and used without restating their proofs. It contributes four things. **(1)** A symmetry-reduced variant **PCF'** of PCF: relax “one configuration per position” to “at most one”, and force the empty positions to a suffix. We prove PCF' is exact and — importantly — that its LP bound is *unchanged* at $|U| - b$: the modification removes equivalent integer solutions (shrinks the branch-and-bound tree) but does *not* strengthen the root relaxation. **(2)** A self-contained **branch-and-price primer**, written for a reader new to column generation. **(3)** The **pricing subproblems** for PCF' and PTF, derived from the LP duals: PCF' prices a single b -subset with job-coverage bonuses (a job-grouping pricing problem); PTF prices a *coupled pair* of b -subsets (bilinear), the algorithmic price of its stronger bound. **(4)** **Implementation instructions** (SCIP/PySCIPOpt first) for a later session. Proofs in Sec. 2 are complete and numerically checked. The PCF' reduced cost (Sec. 4) has been **verified against solver ground truth** on seven instances — a sign error in an earlier draft was caught and corrected. The PTF reduced cost (Sec. 5) has **now also been verified**: its compact arc model is exact ($\text{IP} = Z_{\text{SSP}}^*$) and its printed reduced cost matches solver ground truth on all non- $\perp$ arcs across five instances, with its signs — unlike PCF's — already correct. This is an AI-assisted draft, not verified ground truth (repo norm).

Contents

1	Recap and notation	3
2	PCF': a symmetry-reduced PCF	3
2.1	The formulation	3
2.2	Exactness	4
2.3	The bound does not change	4
2.4	Why each piece is exactly what it is	4
3	Branch-and-price, from the ground up	5
3.1	The problem column generation solves	5
3.2	The mechanics	5
3.3	Dantzig–Wolfe view of PCF' and PTF	6
3.4	From an LP method to an exact integer method	6
3.5	Symmetry, revisited in the CG setting	6
3.6	Why the bound is the whole game	7
4	Pricing subproblem for PCF'	7
4.1	What is generated, what is structural	7
4.2	Duals and the reduced cost of a column	7
4.3	The pricing problem is a b -subset with coverage bonuses	7

5 Pricing subproblem for PTF	8
5.1 The formulation, restated	8
5.2 Duals and the reduced cost of an arc	9
5.3 Pricing is a <i>coupled pair</i> of b -subsets	9
6 Implementation plan (SCIP / PySCIPOpt first)	10
6.1 Why SCIP, not CPLEX, for the prototype	10
6.2 Components (PCF' first)	10
6.3 PTF differences	10
6.4 Phased milestones (each gates the next)	11
6.5 Token / session economy	11
7 Open problems	11

Recap and notation

Jobs $J = \{1, \dots, n\}$ with tool sets $T_j \subseteq T$, $|T_j| \leq b \leq |T|$; $U = \bigcup_j T_j$; configurations $V = \{C \subseteq T : |C| = b\}$; $H_j = \{C \in V : T_j \subseteq C\}$; transition cost $d(C, C') = |C' \setminus C|$. The free-initial-load value Z_{SSP}^* counts insertions after a cost-free first configuration. From Part X we reuse, verbatim, the two formulations and one lemma.

PCF (position–configuration). Binary y_C^k ($C \in V$, $k = 1, \dots, n$): configuration C occupies position k . Continuous $w_t^k \geq 0$ ($t \in T$, $k = 2, \dots, n$): an insertion of tool t at position k . Write $a_t^k := \sum_{C \ni t} y_C^k$ (the “presence mass” of t at position k). The model is

$$\begin{aligned} & \min \sum_{t \in T} \sum_{k=2}^n w_t^k \\ \text{(P)} \quad & \sum_{C \in V} y_C^k = 1 \quad k = 1, \dots, n \end{aligned} \tag{1}$$

$$\text{(C)} \quad \sum_{k=1}^n \sum_{C \in H_j} y_C^k \geq 1 \quad j \in J \tag{2}$$

$$\text{(W)} \quad w_t^k \geq a_t^k - a_t^{k-1} \quad t \in T, k = 2, \dots, n \tag{3}$$

$$\text{(T)} \quad \sum_{k=2}^n w_t^k \geq 1 - a_t^1 \quad t \in U. \tag{4}$$

Part X proves PCF is exact, that its plain LP (dropping (4)) is 0, and that with the *tool-counting rows* (4) the LP equals $|U| - b$.

PTF (position–transition). Binary $z_{(C, C')}^k$ on the diagonal-free arc set $A = \{(C, C') : C \neq C' \in V\} \cup \{(C, \perp) : C \in V\} \cup \{(\perp, \perp)\}$, $k = 1, \dots, n-1$, carries the cost $d(C, C')$ directly; per-tool “head = tail” rows chain the steps. PTF is exact and its LP can strictly exceed $|U| - b$. We restate it in full in Sec. 5.

Normal form (Part X). Every feasible instance has an optimal schedule whose magazine trajectory is a sequence of $m \leq n$ *pairwise distinct* configurations $C_{(1)}, \dots, C_{(m)}$, each serving at least one job, of cost $\sum_{l < m} d(C_{(l)}, C_{(l+1)})$. This lemma is the engine of every exactness argument below.

PCF': a symmetry-reduced PCF

The dominant symmetry of PCF is *padding*. To fill n positions with a length- m trajectory ($m < n$) one inserts $n - m$ repeated configurations, and they may be placed at many positions: the single physical schedule $D_1 D_2 D_3$ has encodings $D_1 D_1 D_2 D_3$, $D_1 D_2 D_2 D_3$, $D_1 D_2 D_3 D_3, \dots$, all of equal cost. A solver explores these as distinct nodes. Sec. 2 removes the padding freedom.

The formulation

PCF'. Same variables as PCF. Replace the assignment equation (1) by an inequality and add a contiguity row:

$$\begin{aligned} & \min \sum_{t \in T} \sum_{k=2}^n w_t^k \\ \text{[P']} \quad & \sum_{C \in V} y_C^k \leq 1 \quad k = 1, \dots, n \end{aligned} \tag{5}$$

$$\text{[G]} \quad \sum_{C \in V} y_C^{k+1} \leq \sum_{C \in V} y_C^k \quad k = 1, \dots, n-1 \tag{6}$$

and (2), (3), (4) unchanged.

Call position k *used* if $\sum_C y_C^k = 1$ and *empty* if $\sum_C y_C^k = 0$. Row count $2n + (n-1)|T| + |U| + (n-1) = O(n|T|)$, still fixed in advance.

Remark 2.1 (reading (6)). At any integer point $\sum_C y_C^k \in \{0, 1\}$, and (6) makes this sequence non-increasing in k : a block of 1s followed by 0s. Hence the used positions are a *prefix* $\{1, \dots, p\}$ and the empties are a *suffix*. The free initial configuration sits at position 1.

Exactness

Proposition 2.2 (PCF' is exact). *The optimum of PCF' with binary y equals Z_{SSP}^* .*

Proof. ($\leq$) Take a normal-form schedule $D_1, \dots, D_m$ ($m \leq n$, distinct). Set $y_{D_l}^l = 1$ for $l = 1, \dots, m$, leave positions $m+1, \dots, n$ empty, and $w_t^k = \max(0, a_t^k - a_t^{k-1})$. Rows (5) hold ($= 1$ on the prefix, $= 0$ after); (6) holds (prefix); (2),(3),(4) hold exactly as in the PCF proof. Objective: for $k = 2, \dots, m$, $\sum_t \max(0, a_t^k - a_t^{k-1}) = d(D_{k-1}, D_k)$; the transition $m \rightarrow m+1$ is full $\rightarrow$ empty, so $a_t^{m+1} - a_t^m = -[t \in D_m] \leq 0$ and contributes 0; later steps contribute 0. The total is $\sum_{l < m} d(D_l, D_{l+1}) = Z_{\text{SSP}}^*$. Hence $\min \leq Z_{\text{SSP}}^*$.

($\geq$) Take any integer-feasible PCF' point. By remark 2.1 the used positions form a prefix $1, \dots, p$, each holding exactly one configuration (integral, $\sum_C y_C^k = 1$ there), giving a trajectory $D_1, \dots, D_p$; empty positions hold no configuration. By (3) the objective is $\geq \sum_{k=2}^p \sum_t \max(0, a_t^k - a_t^{k-1}) = \sum_{k=2}^p d(D_{k-1}, D_k)$, the empty suffix adding 0. By (2) every job is covered by some D_k with $k \leq p$ (empty positions contribute nothing to (2)). Assign each job to a covering position, order jobs by position: the resulting schedule has KTNS cost $\leq$ the trajectory cost $\leq$ the objective, so $Z_{\text{SSP}}^* \leq$ objective. Hence $\min \geq Z_{\text{SSP}}^*$. $\square$

The bound does not change

This is the load-bearing fact for branch-and-price: PCF' is *not* a stronger relaxation than PCF, so the only thing it can buy a tree-search solver is fewer equivalent nodes.

Proposition 2.3 (LP invariance). *Let $\text{LP}(\cdot)$ denote the linear-relaxation value.*

- (a) *The feasible region of PCF is contained in that of PCF'; hence $\text{LP}(\text{PCF}') \leq \text{LP}(\text{PCF})$ for both the plain and the full model.*
- (b) *Without (4), $\text{LP}(\text{PCF}') = 0$.*
- (c) *With (4), $\text{LP}(\text{PCF}') \geq |U| - b$. Consequently $\text{LP}(\text{PCF}') = |U| - b$ on every instance where $\text{LP}(\text{PCF}) = |U| - b$ — i.e. on all instances tested in Part X (reconfirmed for the 6-ring in Sec. 6).*

Proof. (a) Any PCF-feasible point has $\sum_C y_C^k = 1$ for all k , so $\sum_C y_C^k \leq 1$ ((5)) and $\sum_C y_C^{k+1} = 1 \leq 1 = \sum_C y_C^k$ ((6)) hold; the remaining rows are shared. So $\{\text{PCF}\} \subseteq \{\text{PCF}'\}$ and minimising over the larger set can only lower the value.

(b) The PCF plain-LP-zero certificate — pick $C_j \in H_j$ and set $y^k = \frac{1}{n} \sum_j \delta_{C_j}$ at every position, $w = 0$ — has $\sum_C y_C^k = 1$ for all k , hence satisfies (5) and (6); its objective is 0, and $\text{LP} \geq 0$ trivially.

(c) Summing (4) over $t \in U$ and using $\sum_{t \in U} a_t^1 \leq \sum_{t \in T} a_t^1 = \sum_C |C| y_C^1 = b \sum_C y_C^1 \leq b$ (now with “ $\leq b$ ” from (5), not “ $= b$ ”) gives objective $\geq |U| - \sum_{t \in U} a_t^1 \geq |U| - b$ at every PCF' point. With (a), $|U| - b \leq \text{LP}(\text{PCF}') \leq \text{LP}(\text{PCF})$, so equality holds wherever the right side equals $|U| - b$. $\square$

Why each piece is exactly what it is

Remark 2.4 (contiguity (6) is for *correctness*, not decoration). Relaxing (1) to (5) *without* (6) breaks the model. Suppose an empty position sat in the interior, $\dots D_{k-1}, \emptyset, D_{k+1} \dots$. Two failures: the cost row (3) reads the empty $\rightarrow$ full re-entry as $a_t^{k+1} - a_t^k = 1$ for *every* tool of D_{k+1} and charges a spurious full reload of b ; and the counting rows (4), whose free-load term a_t^1 is

pinned to position 1, would mis-attribute the cost-free load if position 1 were empty. Forcing the empties to a suffix (6) keeps the free configuration at position 1 and makes every used-to-used step a genuine transition. So (6) is not optional symmetry-breaking; it is what makes the relaxed model mean what PCF means.

Remark 2.5 (“each configuration at most once” is *not* added as rows). The normal form licenses restricting to schedules whose configurations are pairwise distinct, i.e. $\sum_k y_C^k \leq 1$ for every $C \in V$. We deliberately do *not* impose it, for three reasons. (i) *Coupling*: with the equality (1) still in force it would require n *distinct* configurations to fill n positions — infeasible or strictly suboptimal whenever the optimal trajectory has $m < n$ configurations; it only makes sense *together with* the relaxation (5). (ii) *It destroys the design*: as explicit rows it is one constraint *per configuration*, i.e. $|V| = \binom{|T|}{b}$ rows — exponential — which is exactly the row blow-up that the position formulations were built to avoid. (iii) *It is redundant for optimality*: a non-adjacent repeat $D_1 D_2 D_1 D_3$ re-pays the return cost $d(D_2, D_1) \geq 0$ and is dominated, so an optimal solution never uses one. The pair (5)+(6) already gives each trajectory a *unique* encoding (prefix $D_1 \cdots D_m$, then empties). If distinctness is ever wanted at a node it is enforced by *branching* (Sec. 3), never by adding the rows.

Remark 2.6 (what is bought, and what is not). (5)+(6) collapse the padding orbit to one representative, which removes equivalent nodes from a branch-and-bound / branch-and-price tree. By proposition 2.3 they do *not* change the root bound $|U| - b$. The bound weakness is a *different* defect, addressed only by a stronger relaxation (PTF, or new valid inequalities — Part X OP-X2). Keep the two concerns separate: PCF' attacks symmetry; PTF attacks the bound.

Verified (`verify_pcf_prime.py`, full-enumeration MILP via PuLP/CBC, 2026-06-22). On the 6-ring ($b=3$, $|U| - b = 3$, brute-force KTNS $Z_{\text{SSP}}^* = 3$): for *both* PCF and PCF', plain LP = 0.000, LP with (4) = 3.000 = $|U| - b$, and IP = 3.000 = Z_{SSP}^* . PCF' reproduces PCF on every metric, confirming proposition 2.2 and proposition 2.3: exactness holds and the relaxation is unchanged.

Branch-and-price, from the ground up

This section assumes no prior exposure to column generation. The reader who knows Barnhart et al. [1998] and Lübbecke and Desrosiers [2005] can skip to Sec. 4.

The problem column generation solves

Both PCF' and PTF have a *polynomial* number of rows but an *exponential* number of columns: one y_C^k per configuration $C \in V$ (PCF'), or one $z_{(C,C')}^k$ per arc (PTF), and $|V| = \binom{|T|}{b}$ is astronomically large. We cannot even write the LP down. Column generation (CG) solves such an LP *without enumerating its columns*.

The mechanics

Work with a *restricted master problem* (RMP): the same rows, but only a small subset Ω of the columns. Then iterate:

1. Solve the RMP *linear relaxation*. This yields a primal solution and, crucially, a vector of *dual values* (one per row).

2. *Pricing*. A column not in Ω can improve the LP only if its *reduced cost* is negative. For a minimisation with columns at lower bound 0,

$$\bar{c}(\text{column}) = (\text{objective coefficient}) - \sum_{\text{rows } r} \pi_r (\text{coefficient of the column in row } r),$$

where π_r are the duals from step 1. Instead of scanning all columns we *optimise*: find the single column of *minimum* reduced cost by solving a *pricing subproblem* over the implicit description of a column (here: choose a configuration, or an arc).

3. If $\min \bar{c} \geq 0$, no column can improve the LP: the RMP LP value equals the *full* LP value and CG stops. Otherwise add the offending column(s) to Ω and return to step 1.

The art is entirely in step 2: the pricing subproblem must (a) have the same optimum as a brute scan over all columns, and (b) be solvable quickly. Sections 4 and 5 derive it for our two models.

Dantzig–Wolfe view of PCF' and PTF

CG is the algorithmic face of a Dantzig–Wolfe reformulation [Vanderbeck and Wolsey, 2010]: the “hard” combinatorial object — the configuration set V — is pushed into the columns, while the coupling constraints (coverage, counting, chaining) stay in the master as the fixed $O(n|T|)$ rows. This is precisely the property Part X engineered: *rows fixed in advance, columns generated on demand*. A generated configuration (PCF') or arc (PTF) drops into existing rows and never creates a new one. Historically this is the same move Crama and Oerlemans [1994] used to price job groups for flexible manufacturing systems; our pricing problems are close relatives.

From an LP method to an exact integer method

CG solves one LP — the relaxation at one node. To obtain an *integer* optimum we run branch-and-bound and solve *every node's* LP by column generation. That is *branch-and-price* [Barnhart et al., 1998]. The one genuinely new difficulty is that branching decisions must be compatible with pricing:

- **Naïve branching on a single column** ($y_C^k = 0$ or 1). Fixing $y_C^k = 0$ *forbids* a column; the pricer must then be prevented from regenerating it, so the pricing oracle has to carry a growing list of forbidden columns. Fixing variables this way also yields lopsided, deep trees. Usable but clumsy.
- **Robust branching** (recommended). Branch on an *original*, polynomially-many quantity whose effect on the pricer is merely a change of one dual. For PCF' the natural choice is the *tool-presence* variable $a_t^k = \sum_{C \ni t} y_C^k \in \{0, 1\}$: “is tool t in the magazine at position k ?”. Branching $a_t^k = 0 / 1$ adds one bound; in the pricer it just shifts the weight ρ_t^k of tool t at position k (Sec. 4), leaving the oracle's *structure* untouched. “Robust” means exactly this: the subproblem keeps its form.
- **Ryan–Foster branching** [Ryan and Foster, 1981]. For set-partition-flavoured masters, branch on whether two elements are forced *together* or *apart* (here: two jobs sharing a position, or a job pinned to/away from a position). It partitions the LP feasible set cleanly and is pricing-compatible. A standard fallback when single-bound branching stalls.

Symmetry, revisited in the CG setting

The position index is the source of symmetry. Three points decide how to treat it under CG. First, the contiguity rows (6) are *master* rows in the original variables ($n - 1$ of them); they are fully CG-compatible — they simply contribute $n - 1$ extra duals to the reduced cost. Keep them. Second, the “at-most-once” rows are *not* CG-compatible (exponentially many, one per configuration); do not add them (remark 2.5). Third, piling on further static lexicographic order cuts tends to *fight* the pricer, because they impose an ordering the subproblem does not respect.

The textbook resolution [Vanderbeck and Wolsey, 2010, Lübbecke and Desrosiers, 2005] is that the Dantzig–Wolfe reformulation already aggregates identical subproblems, and any *residual* symmetry is broken in the *branching rule* (branch on a_t^k , or Ryan–Foster), not by more cuts. **Recommendation:** keep (6); handle the rest by branching.

Why the bound is the whole game

Branch-and-bound effort grows with the *integrality gap* $Z_{\text{SSP}}^* - \text{LP}_{\text{root}}$. Here PCF' has $\text{LP}_{\text{root}} = |U| - b$, which can sit far below Z_{SSP}^* , so its tree can be large; PTF has $\text{LP}_{\text{root}} > |U| - b$, a smaller tree, but a harder pricing subproblem (Sec. 5). *Stronger bound vs. cheaper pricing* is the central engineering trade-off of this whole programme. (The 6-ring is deceptively kind: there $|U| - b = 3 = Z_{\text{SSP}}^*$, so even PCF' has a zero root gap. General instances do not.)

Pricing subproblem for PCF'

What is generated, what is structural

The continuous insertion variables w_t^k number $(n-1)|T| = O(n|T|)$ — polynomially many — so they live in the master at all times. The exponential family is $\{y_C^k\}$: *these are the generated columns*, one per (configuration C , position k).

Duals and the reduced cost of a column

Attach a dual to each master row of the PCF' LP: α_k to (5) (≤ 0), γ_k to (6) (≤ 0), $\pi_j \geq 0$ to coverage (2), $\mu_{t,k} \geq 0$ to the cost rows (3), and $\lambda_t \geq 0$ to the counting rows (4). Collecting the coefficient of a single column y_C^k in every row (it has objective coefficient 0):

- (5) at position k : coefficient $+1$.
- (6): y_C^k appears in row $k-1$ as the $+\sum_C y_C^k$ term (coeff $+1$, for $k \geq 2$) and in row k as the $-\sum_C y_C^k$ term (coeff -1 , for $k \leq n-1$).
- coverage (2): coefficient $+1$ for every job j with $T_j \subseteq C$ (i.e. $C \in H_j$).
- cost rows (3): through $a_t^k = \sum_{C \ni t} y_C^k$, the column contributes to $-a_t^k$ in row (t, k) (coeff -1 , each $t \in C$, $k \geq 2$) and to $+a_t^k$ in row $(t, k+1)$ (coeff $+1$, each $t \in C$, $k \leq n-1$).
- counting rows (4): only the $+a_t^1$ term, so only at $k=1$ (coeff $+1$, each $t \in C \cap U$).

Hence the reduced cost is

$$\bar{c}(C, k) = \underbrace{-\alpha_k - \gamma_{k-1}[k \geq 2] + \gamma_k[k \leq n-1]}_{\beta_k \text{ (depends only on } k)} - \sum_{t \in C} \rho_t^k - \sum_{j: T_j \subseteq C} \pi_j, \quad (7)$$

where the per-tool, per-position weight is

$$\rho_t^k := -\mu_{t,k}[k \geq 2] + \mu_{t,k+1}[k \leq n-1] + \lambda_t[k=1][t \in U]. \quad (8)$$

(Verified & corrected 2026-06-22, verify_pricing.py: this reduced cost was checked against CBC's ground-truth reduced costs on the 6-ring and six random small instances, matching to $< 10^{-9}$ over every column. An earlier draft had the three per-tool signs in (8) flipped (a per-column error of 4–6); the signs above are the corrected, verified ones. The position constant β_k and the additive structure — all the pricing oracle actually needs — were confirmed correct.)

The pricing problem is a b -subset with coverage bonuses

By (7), for a *fixed* position k the most negative reduced cost is obtained by *maximising* the configuration-dependent part:

$$\max_{C \subseteq T, |C|=b} \sum_{t \in C} \rho_t^k + \sum_{j: T_j \subseteq C} \pi_j \quad (9)$$

and a column prices in iff $\beta_k - (\text{this max}) < 0$ for some k . In words: choose b tools to collect tool rewards ρ_t^k *plus* a bonus $\pi_j \geq 0$ for every job whose *entire* tool set lands inside C . This is the job-grouping pricing class — exactly the “single set with job-coverage bonuses” of Part X.

Exact MILP. Binary $x_t = [t \in C]$ and $u_j = [T_j \subseteq C]$:

$$\max \sum_{t \in T} \rho_t^k x_t + \sum_{j \in J} \pi_j u_j \quad \text{s.t.} \quad \sum_t x_t = b, \quad u_j \leq x_t \quad (\forall t \in T_j), \quad x_t \in \{0, 1\}, \quad u_j \in [0, 1].$$

Because $\pi_j \geq 0$, at the optimum $u_j = \min_{t \in T_j} x_t = [T_j \subseteq C]$ automatically, so u_j may stay continuous. Solve once per position k (or jointly with k as an index).

Complexity and shortcuts. The bonus term makes (9) a budgeted maximum-coverage variant — NP-hard in b in general — but for *fixed* b it is polynomial: enumerate the $\binom{|T|}{b}$ candidate subsets. At SSP magazine sizes this is milliseconds, and a closed enumeration is often faster than invoking a MILP solver. In production: a greedy heuristic pricer (add the tool of best marginal reward, repair to $|C'| = b$) returns columns cheaply, with the exact MILP/enumeration run *only* when the heuristic finds none — the exact solve is mandatory before declaring CG converged.

Branching link. Branching on a_t^k (Sec. 3) injects its branching dual additively into ρ_t^k in (8); the pricer (9) is unchanged in form. That is what makes the scheme robust.

Verified (verify_pricing.py, 2026-06-22). The reduced cost (7) with the corrected weights (8) reproduces the ground-truth reduced cost (computed definitionally from the model duals, and cross-checked against CBC’s own $\bar{c}_j$) on the 6-ring and six random instances ($J \leq 6$, $|T| \leq 7$, $b \in \{2, 3\}$): maximum discrepancy $< 10^{-9}$ over *all* columns, while complementary slackness ($\bar{c} \geq 0$ everywhere, $= 0$ on basic columns) independently confirms the dual extraction. The previously printed signs gave a discrepancy of 4–6 per column — exactly the class of bug this check exists to catch, and the reason PTF’s (15) is not trusted until its own run.

Pricing subproblem for PTF

The formulation, restated

With $\text{hd}_t^k = \sum_{(C, C') \in A: t \in C'} z^k$, $\text{tl}_t^k = \sum_{(C, C') \in A: t \in C} z^k$, and $\text{ins}_t^k = \sum_{(C, C') \in A: t \in C' \setminus C} z^k$ ($t \in \perp$ always false), PTF is

$$\begin{aligned} \min \quad & \sum_{k=1}^{n-1} \sum_{(C, C') \in A} d(C, C') z_{(C, C')}^k \\ \text{(Pz)} \quad & \sum_{(C, C') \in A} z_{(C, C')}^k = 1 \end{aligned} \quad \begin{aligned} & k = 1, \dots, n-1 \end{aligned} \quad (10)$$

$$\text{(cons)} \quad \text{hd}_t^k = \text{tl}_t^{k+1} \quad t \in T, \quad k \leq n-2 \quad (11)$$

$$\text{(abs)} \quad \sum_{(C, \perp) \in A} z^k \leq z_{(\perp, \perp)}^{k+1} \quad k \leq n-2 \quad (12)$$

$$\text{(Cz)} \quad \sum_{T_j \subseteq C} z_{(C, C')}^1 + \sum_k \sum_{T_j \subseteq C'} z_{(C, C')}^k \geq 1 \quad j \in J \quad (13)$$

$$\text{(Tz)} \quad \sum_{k=1}^{n-1} \text{ins}_t^k \geq 1 - \text{tl}_t^1 \quad t \in U. \quad (14)$$

The columns are arcs $z_{(C, C')}^k$; there is no separate w . Compared with PCF', two things change the pricing: the cost sits *on the column* ($d(C, C') = |C' \setminus C|$), and the chaining rows (11) couple a column’s *head* tools to the next step’s *tail*.

Duals and the reduced cost of an arc

Duals: σ_k (free) on (10); $\delta_{t,k}$ (free) on (11); $\phi_k \leq 0$ on (12); $\pi_j \geq 0$ on (13); $\lambda_t \geq 0$ on (14). For a non- $\perp$ arc $z_{(C,C')}^k$ (objective coefficient $d(C,C') = \sum_t [t \in C'] [t \notin C]$), collecting coefficients:

- (10) at k : $+1 \Rightarrow \sigma_k$.
- (11): this column is the *head* of step k , so $+1$ in row (t, k) for each $t \in C'$ (if $k \leq n-2$); and the *tail* of step k , so -1 in row $(t, k-1)$ for each $t \in C$ (if $2 \leq k \leq n-1$).
- coverage (13): at $k=1$, $+1$ for jobs with $T_j \subseteq C$ (first tail); at any k , $+1$ for jobs with $T_j \subseteq C'$ (head).
- counting (14): $+\lambda_t$ for each $t \in C' \setminus C$ (the ins term), and at $k=1$ additionally $+\lambda_t$ for each $t \in C$ (the tl_t^1 term).

Therefore

$$\begin{aligned} \bar{c}(C, C', k) = & \sum_t [t \in C' \setminus C] (1 - \lambda_t) - \sigma_k - \sum_{t \in C'} \delta_{t,k} [k \leq n-2] + \sum_{t \in C} \delta_{t,k-1} [2 \leq k \leq n-1] \\ & - \sum_{j: T_j \subseteq C'} \pi_j - [k=1] \sum_{j: T_j \subseteq C} \pi_j - [k=1] \sum_{t \in C} \lambda_t. \end{aligned} \quad (15)$$

(**Verified** 2026-06-22, `verify_ptf_pricing.py`: unlike PCF', the printed signs of (15) are correct — it reproduces CBC's ground-truth reduced costs on all non- $\perp$ arcs over the 6-ring and four random instances, exact to $< 10^{-9}$. $\perp$ -arcs $z_{(C,\perp)}^k, z_{(\perp,\perp)}^k$ have $d=0$ and only the $\sigma, \phi, \delta, \pi$ contributions of their single real endpoint — a handful of cheap columns priced by inspection, confirmed to price ≥ 0 at the LP optimum.)

Pricing is a coupled pair of b -subsets

The decisive term is $\sum_t [t \in C' \setminus C] (1 - \lambda_t)$: it is *bilinear* in the indicator of C and the indicator of C' . Unlike PCF', the pricing problem cannot separate into one set — it must choose two b -subsets $C \neq C'$ at once, with the cost and the ins reward both living on the *difference* $C' \setminus C$. This is the algorithmic cost of PTF's stronger LP bound.

Linearised MILP (per step k). Binaries $x_t = [t \in C]$, $x'_t = [t \in C']$, and a linearisation $p_t = [t \in C' \setminus C] = x'_t(1 - x_t)$ via the standard McCormick inequalities:

$$\begin{aligned} \min \quad & \sum_t (1 - \lambda_t) p_t - \sigma_k - \sum_t \delta_{t,k} [k \leq n-2] x'_t + \sum_t \delta_{t,k-1} [2 \leq k \leq n-1] x_t \\ & - \sum_j \pi_j u'_j - [k=1] \sum_j \pi_j u_j - [k=1] \sum_t \lambda_t x_t \\ \text{s.t.} \quad & \sum_t x_t = b, \quad \sum_t x'_t = b, \quad p_t \leq x'_t, \quad p_t \leq 1 - x_t, \quad p_t \geq x'_t - x_t, \quad p_t \geq 0, \\ & u'_j \leq x'_t \ (\forall t \in T_j), \quad u_j \leq x_t \ (\forall t \in T_j), \quad \sum_t p_t \geq 1 \quad (\text{enforces } C \neq C'), \\ & x_t, x'_t \in \{0, 1\}, \quad u_j, u'_j \in [0, 1]. \end{aligned}$$

Here $u'_j = [T_j \subseteq C']$, $u_j = [T_j \subseteq C]$, and $\sum_t p_t = d(C, C') \geq 1$ excludes the diagonal. The model has $2|T|$ binaries plus $|T|$ linearisation variables — strictly larger and harder than the PCF' pricer (9).

Heuristic pricing. Alternate: fix C (e.g. from a PCF'-style single-set pricer) and optimise C' (now a pure b -subset problem), then fix C' and optimise C ; iterate a few rounds. Use the exact coupled MILP only as the convergence-guaranteeing fallback. Degeneracy from the $n-1$ identical step rows (10) should be expected; stabilisation (Lübbecke and Desrosiers, 2005) or dual smoothing helps.

Verified (`verify_ptf_pricing.py`, 2026-06-22). *Model first*: the full-arc PTF MILP returns $IP = Z_{SSP}^*$ on the 6-ring and four random instances (brute-force KTNS), so the diagonal-free/absorbing formulation is implemented faithfully. *Then the reduced cost*: the printed (15) reproduces the ground-truth reduced cost (definitional, and CBC's $\bar{c}_j$, themselves agreeing to 8×10^{-15}) on *every* non- $\perp$ arc across all five instances — maximum discrepancy 0 — with optimality ($\bar{c} \geq 0$, $= 0$ on basic arcs, $\perp$ -arcs included) holding throughout. So, in contrast to PCF' (eq. (8)), PTF's printed signs needed no correction. On the 6-ring the PTF LP equals $3 = Z_{SSP}^*$ (tight), consistent with Part X.

Implementation plan (SCIP / PySCIPOpt first)

We will build this together in a later session; this section fixes the design so that session is mechanical.

Why SCIP, not CPLEX, for the prototype

SCIP exposes a first-class *pricer* plugin: in PySCIPOpt one subclasses `Pricer` and implements `pricerredcost` (reduced-cost pricing) and `pricerfarkas` (Farkas pricing for infeasible RMPs); SCIP runs the branch-and-bound and calls back for columns. GCG — the generic branch-price-and-cut solver built on SCIP — goes further and can detect a Dantzig–Wolfe structure automatically, letting us test the reformulation with almost no pricing code. CPLEX has *no* native column-generation loop: one would drive branch-and-bound through the generic-callback interface and manage the RMP and pricing by hand — substantially more plumbing and easy to get subtly wrong. Plan: prototype and validate in SCIP/PySCIPOpt; only port the winner to CPLEX if we need parity with the existing BBC timings (CLAUDE.md: CPLEX is the repo's primary solver, all baselines thread-pinned).

Components (PCF' first)

1. **Instance load.** `utils.load_ssp_instance(path) → (J, T, C, A, T_j)`; start with `data/Shankar/shankar-` (the 6-ring).
2. **Restricted master.** Rows (5),(6),(2),(3), (4); structural w_t^k ; seed columns from a feasible schedule (`heuristics.warmstart_from_jgp` or `greedy_ffd` → a job sequence → its per-position configurations). A feasible seed (covering every job) gives valid duals immediately and avoids leaning on Farkas pricing early.
3. **Pricer.** In `pricerredcost`: read $\alpha, \gamma, \pi, \mu, \lambda$; form ρ_t^k (8) and β_k ; for each position k solve (9) (enumeration for small b , else the MILP); add every column with $\bar{c} < -\varepsilon$. Implement `pricerfarkas` too, or guarantee initial feasibility so it is trivial.
4. **Branching.** Begin with SCIP's default on the a_t^k presence variables (introduce them as bounded auxiliaries tied to y through the existing (3) linkage, or via a small constraint handler); escalate to Ryan–Foster only if needed.
5. **Extract and validate.** On an integral leaf, read the prefix trajectory $D_1, \dots, D_p$, flatten to a schedule, and assert `compute_ktns(sequence) = objective`. Mismatch $\Rightarrow$ a modelling or sign bug, stop.

PTF differences

Columns are arcs; the pricer solves the coupled-pair MILP of Sec. 5; chaining (11) and absorption (12) are ordinary master rows. Expect slower pricing and more degeneracy; budget for stabilisation.

Phased milestones (each gates the next)

- P0 — compact sanity [DONE 2026-06-22].** *Verified* by `verify_pcf_prime.py`, `verify_pricing.py`, `verify_ptf_pricing.py` (in `plans-genai/_verification/`); a sign bug in PCF' pricing was caught and fixed. Build PCF' as a *full* MILP with *all* $\binom{|T|}{b}$ configurations enumerated (only tiny $|T|$), on the 6-ring and a few random instances. Assert $IP = Z_{SSP}^*$ (cross-check `compute_ktns` / brute force), plain LP = 0, LP with (4) = $|U| - b$. This validates propositions 2.2 and 2.3 *before* any CG code exists.
- P1 — PCF' CG at the root.** Implement the pricer; assert the CG root LP equals the P0 compact LP. This is the *sign check* for (7)–(8): if the reduced-cost algebra is wrong, the two LP values diverge.
- P2 — PCF' full branch-and-price.** Add branching; assert the integer optimum equals brute force / the BBC solver on the 6-ring and randoms; log node counts against a plain compact PCF solve to quantify the symmetry win.
- P3 — PTF.** Repeat P1–P2 with the coupled pricer; confirm the root LP exceeds $|U| - b$ on the Part X “above-bounds” instance.
- P4 — benchmark.** Compare against BBC / SSPMF on a Tabela1C subset, recording `obj_ktns` (empty-start) per the repo convention, threads pinned.

Token / session economy

Load the `ssp-bbc-expert` skill at the start of the implementation session (it carries the CPLEX/repo context); keep `verify_pcf_prime.py` and the pricer in `src/BNP/` next to the existing solvers; and run the P0/P1 verification in an isolated subagent so its solver output does not consume the main session's context.

Open problems

OP-X5 (pricing complexity). The PCF' pricer (9) is polynomial for fixed b (enumeration) but a budgeted-coverage problem in general. Is it NP-hard as a function of b ? Characterise its relation to the job-grouping pricing of Crama and Oerlemans [1994], and decide whether a combinatorial (non-MILP) oracle exists at the magazine sizes of the Tabela1C benchmarks.

OP-X6 (pricing-compatible symmetry breaking). Prove that branching on a_t^k (or a Ryan–Foster rule on job/position pairs) preserves both pricing oracles *and* eliminates the residual position symmetry not already killed by (6); quantify the interaction between (6) and the branching rule (do they ever over-constrain jointly?).

OP-X7 (does the stronger bound pay?). Empirically, net of pricing cost, does PTF's stronger root LP (Sec. 5, Part X) yield smaller branch-and-price trees than PCF', or does the coupled-pair pricer erase the advantage? Settle on the Tabela1C subset and on the disjoint-ring family (Part V) where the gap grows.

References

-
- C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.
- Y. Crama and A. G. Oerlemans. A column generation approach to job grouping for flexible manufacturing systems. *European Journal of Operational Research*, 78(1):58–80, 1994.
- M. E. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.
- D. M. Ryan and B. A. Foster. An integer programming approach to scheduling. *Computer Scheduling of Public Transport*, pages 269–280, 1981.
- F. Vanderbeck and L. A. Wolsey. Reformulation and decomposition of integer programs. In M. Jünger et al., editors, *50 Years of Integer Programming 1958–2008*, pages 431–502. Springer, Berlin, Heidelberg, 2010.